TU Wien
Institut für Logic and Computation
Forschungsbereich Algorithms and Complexity

186.866 Algorithmen und Datenstrukturen VU

Programmieraufgabe P2

PDF erstellt am: 24. März 2026

1 Vorbereitung

Um diese Programmieraufgabe erfolgreich durchführen zu können, müssen folgende Schritte umgesetzt werden:

1. Laden Sie das zugehörige Java-Projekt `P2.zip` aus TUWEL runter.
2. Entpacken Sie `P2.zip` und öffnen Sie das entstehende Projektverzeichnis, d.h. das *äußere* Verzeichnis `task`, in Ihrer Entwicklungsumgebung.
3. Öffnen Sie die folgende Datei in Ihrem Editor: `task/src/main/java/StudentSolutionForExerciseImplementation.java`
   In dieser Datei sind sämtliche Programmiertätigkeiten durchzuführen. Ändern Sie keine anderen Dateien im Projekt und fügen Sie auch keine neuen hinzu.
4. Füllen Sie Vorname, Nachname und Matrikelnummer in der Methode `StudentInformation provideStudentInformation()` aus.
5. Gehen Sie sicher, dass Java-Version 21 auf Ihrem System verfügbar ist.

   Alternativ zu einer manuellen Installation, können Sie in der Datei `settings.gradle` das auskommentierte Code-Snippet für eine automatische Installation nutzen.
6. Details zur Ausführung des Codes finden Sie in Abschnitt 6.

2 Hinweise

Einige Hinweise, die Sie während der Umsetzung dieser Aufgabe beachten müssen:

- Lösen Sie die Aufgaben selbst. Verwenden Sie auch keine (KI-)Tools z. B. zur Codegenerierung.
- Sie dürfen beliebig viele Hilfsmethoden schreiben und benutzen. Beachten Sie aber, dass Sie nur die oben geöffnete Datei abgeben und diese Datei mit dem zur Verfügung gestellten Framework lauffähig sein muss.
- Es ist grundsätzlich erlaubt, Bibliotheken wie z. B. das Java Collections Framework und die Java Math Klasse zu verwenden. Die Voraussetzung dabei ist, dass die Verwendung die eigentliche Programmieraufgabe nicht abnimmt. Beachten Sie, dass nur die oben genannte Java-Version auf unserem Testsystem zur Verfügung steht.

3 Übersicht

Das Problem, die Knoten eines ungerichteten Graphen mit der minimalen Anzahl an Farben so einzufärben, dass je zwei adjazente Knoten verschiedene Farben besitzen, ist äußerst berühmt. Im Fall von planaren Graphen (wie es etwa die Nachbarschaftsbeziehungen von Ländern auf Landkarten sind) wurde der gewichtige Vier-Farben-Satz[1] erst nach über 100 Jahren vollständig bewiesen. Die sogenannte *chromatische Zahl* eines Graphens gibt diese minimale Anzahl an Farben an. Wir betrachten eine Variante beziehungsweise Verallgemeinerung, bei der eine endliche Menge an Farben fixiert ist und die Anzahl der Verstöße – also die Anzahl an Kanten in einem Graphen, deren Endknoten gleich gefärbt sind – minimiert werden soll.

4 Theorie

Die Theorie für diese Programmieraufgabe befindet sich in den Vorlesungsfolien „Graphen“ und „Greedy-Algorithmen“.

[1] `https://de.wikipedia.org/wiki/Vier-Farben-Satz`

5 Implementierung

Bei dieser Programmieraufgabe werden Sie mit der Klasse `Graph` arbeiten. `Graph` repräsentiert einfache ungerichtete Graphen. `Graph G` bietet folgende Methoden an, die für Ihre Implementierung hilfreich sein könnten:

- `int numberOfVertices()`: Gibt die Anzahl an Knoten zurück. Mittels `int number = G.numberOfVertices()` können Sie die Anzahl der Knoten im Graph `G` auslesen.
- `int[] getVertices()`: Gibt die Knoten des Graphen in aufsteigender Sortierung zurück. Die Knoten des Graphen können mit `int[] vertices = G.getVertices()` abgefragt werden. Die Knoten werden als Array von Knoten-IDs repräsentiert.
- `int[][] getEdges()`: Gibt die Kanten des Graphen zurück. Die Kanten des Graphen können mit `int[][] edges = G.getEdges()` abgefragt werden. Die Kanten werden als Array von Arrays (das innere enthält jeweils genau zwei Knoten-IDs) repräsentiert.
- `int degree(int vertexId)`: Gibt den Knotengrad eines Knotens zurück. Der Knotengrad des Knotens mit der ID 1 kann mittels `int deg = G.degree(1)` ermittelt werden. Bei der Übergabe einer ungültigen Knoten-ID wird `-1` zurückgegeben.
- `int[] getNeighbors(int vertexId)`: Gibt die Nachbarn eines Knotens potentiell unsortiert zurück. Die Nachbarn des Knotens mit der ID 1 können mittels `int[] neighbors = G.getNeighbors(1)` ermittelt werden. Die Nachbarn werden als Array von Knoten-IDs repräsentiert. Bei der Übergabe einer ungültigen Knoten-ID wird `null` zurückgegeben.

5.1 Erster Greedy-Ansatz

Gegeben sei ein ungerichteter Graph $G = (V, E)$ und eine k-elementige Menge von Farben $\{0, 1, \ldots, k-1\}$.

Das Problem der *minimalen chromatischen Verstöße* (MCV) fragt nach einer Farbenzuordnung $c : V \to \{0, 1, \ldots, k-1\}$, sodass die Menge aller Kanten mit chromatischen Verstößen minimal mögliche Größe besitzt. Ein Verstoß ist hier für eine Kante $e = (u, v) \in E$ gegeben, wenn $c(u) = c(v)$.

Schreiben Sie eine Methode `int countChromaticViolations(Graph G, int[] color)`, welche zu einem gegebenen Graphen $G = (V, E)$ mit Knoten indiziert durch `1`,`2`,...,`G.numberOfVertices()` die Kanten in E zählt, deren Endknoten gleich gefärbt sind. Die Farbe des Knotens i ist hierbei im Parameter `int[] color` an Position mit Index $i - 1$ gespeichert.

Wir beschreiben im Folgenden einen Greedy-Algorithmus zur Berechnung einer Farbenzuordnung c für einen gegebenen Graphen G und Farben indiziert durch $\{0, 1, \ldots, k-1\}$. Anfangs haben alle Knoten noch keinen gesetzten Farbwert. Iterativ suchen wir unter allen Knoten mit ungesetztem Farbwert jenen heraus, der größtmöglichen Grad hat (trifft diese Eigenschaft auf mehrere Knoten zu, so wählen wir jenen mit kleinstem Index). Dieser erhält jene Farbe zugewiesen, welche die kleinstmögliche Anzahl an Farbverstößen auslöst (trifft diese Eigenschaft auf mehrere Farben zu, so wählen wir jene mit kleinstem Index). Das Prozedere wird so lange fortgesetzt, bis es keine Knoten mit ungesetzter Farbe mehr gibt; dann terminiert der Algorithmus. Anschließend soll die gefundene Farbzuordnung im Argument `colorSeq` abgespeichert und die Anzahl der Farbverstöße zurückgegeben werden.

Schreiben Sie dazu eine Methode `int greedyMCV(Graph G, int k, int[][] colorSeq)`, die einen Eingabegraphen `Graph G`, eine Farbanzahl `int k` sowie ein bei Aufruf mit `null` initialisiertes Array `int[][] colorSeq` der Größe `G.numberOfVertices()` bestehend aus Arrays der Länge 2 annimmt und zuvor beschriebene Greedy-Prozedur ausführt. Farbzuordnungen sollen hierbei chronologisch direkt in `colorSeq` geschrieben werden. Hier steht `colorSeq[i][0]` für den Knoten, der in Iteration `i` vom Algorithmus ausgewählt wurde, während `colorSeq[i][1]` für die dem Knoten zugewiesene Farbe steht.

5.2 Alternativer Ansatz

Betrachten Sie nun folgende alternative Greedy-Heuristik.

Anfangs haben alle Knoten noch keinen gesetzten Farbwert. Iterativ suchen wir unter allen Knoten mit ungesetztem Farbwert jenen heraus, der größtmöglichen Grad besitzt (trifft diese Eigenschaft auf mehrere Knoten zu, so wählen wir jenen mit kleinstem Index). Für den ausgewählten Knoten wird anschließend jene Farbe bestimmt, die gemäß dem

First-Fit-Prinzip vergeben wird: Wir betrachten die Farben in aufsteigender Reihenfolge und wählen die erste Farbe aus, die bei Zuweisung an den Knoten keinen Farbverstoß mit bereits gefärbten Nachbarn erzeugt. Existiert keine solche farbverstoßfreie Farbe, so wird die Farbe mit dem kleinsten Index gewählt. Das Prozedere wird so lange fortgesetzt, bis es keine Knoten mit ungesetzter Farbe mehr gibt; dann terminiert der Algorithmus.

5.3 Kombination von Greedy und Enumeration

Sie sollen folgende „vorausschauende“ Greedy-Variante implementieren:

1. Anfangs sind alle Knoten ungefärbt.
2. Iterativ wird folgender Schritt bestehend aus (2a)–(2e) ausgeführt (sofern es überhaupt noch ungefärbte Knoten in G gibt, andernfalls wird terminiert):
   (a) Es wird die Menge von Szenarien
   $$S := \{(v, x) : v \in V \text{ ist ungefärbt, } x \text{ ist Farbe in } \{0, 1, \ldots, k-1\}\}$$
   vorbereitet.
   (b) Für jedes Szenario (v, x) wird folgende Subprozedur ausgeführt: In einer frischen Arbeitskopie des partiell gefärbten Graphen wird nun der ungefärbte Knoten v auf x gefärbt; anschließend wenden wir dieselbe Vorgangsweise wie beim zuvor entwickelten Algorithmus `greedyMCV` auf die aktualisierte Version der Arbeitskopie an.
   (c) Das Szenario (v_*, x_*), das zum kleinsten Wert $N_{(v_*,x_*)}$ unter allen Szenarien führt, speichern wir uns ab; ist (v_*, x_*) nicht eindeutig, so wählen wir das Szenario mit kleinstem Knotenindex (und falls immer noch nicht eindeutig, jenes mit kleinstem Farbwert).
   (d) Wir setzen die Farbe von v_* auf x_* und machen damit einen weiteren Schritt in Richtung vollständige Färbung.
   (e) Alle Arbeitskopien können verworfen werden beziehungsweise sind von nun an irrelevant.

Der beschriebene Algorithmus soll in der Methode
```
int greedyLookAheadMCV(Graph G, int k, int[][] colorSeq)
```
implementiert werden.

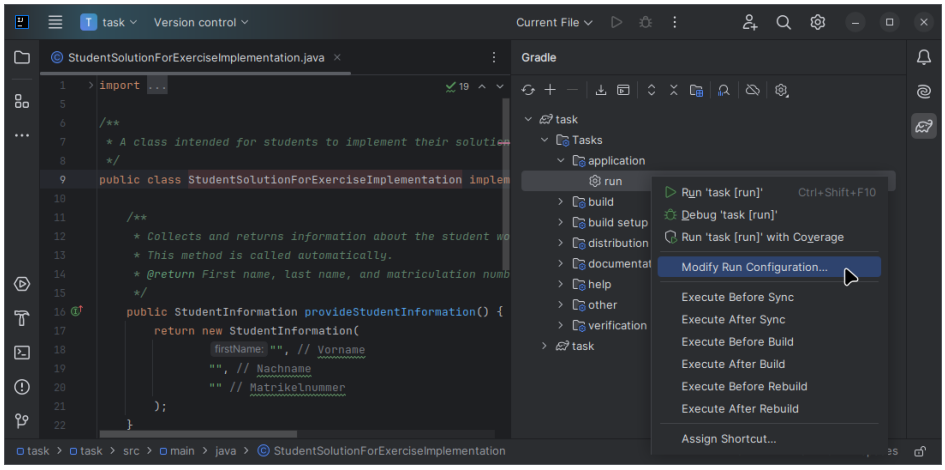

Abbildung 1: Ausführen von Gradle-Tasks in IntelliJ

6 Testen

Führen Sie im Projektverzeichnis den Befehl `./gradlew run` aus (`.\gradlew.bat run` unter Windows).

Sollten Sie IntelliJ nutzen, können Sie das Projekt auch direkt über die grafische Oberfläche starten. Klicken Sie dazu in der rechten Leiste auf das Gradle-Logo (Elefant) und navigieren Sie zum „run“-Task (siehe Abbildung 1). Mittels Rechtsklick bzw. Sekundärklick können Sie das Projekt ausführen („Run“) oder auch den visuellen Debugger nutzen („Debug“). Die Option „Modify Run Configuration...“ erlaubt es Ihnen, die Ausführung permanent als „Run/Debug Configuration“ zu hinterlegen.

Anschließend wird Ihnen in der Konsole eine Auswahl an Testinstanzen angeboten, darunter befindet sich zumindest `abgabe.csv`:

```
Select an instance set or exit:
[1] abgabe.csv
[0] Exit
```

Durch die Eingabe der entsprechenden Ziffer kann entweder eine Testinstanz ausgewählt werden oder das Programm (mittels der Eingabe von `0`) verlassen werden. Wird eine Testinstanz gewählt, dann wird der von Ihnen implementierte Programmcode ausgeführt. Kommt es dabei zu einem Fehler, wird ein Hinweis in der Konsole ausgegeben.

Um nur eine bestimmte Subinstanz auszuführen, hängen Sie `:[Subinstanznummer]` an Ihre Auswahl an (z. B. `1:123`).

Relevant für die Abgabe ist das Ausführen der Testinstanz `abgabe.csv`.

Die Testinstanz `abgabe.csv` sollte in höchstens ein paar Minuten durchlaufen; andernfalls ist etwas ineffizient implementiert.

7 Evaluierung

Wenn der von Ihnen implementierte Programmcode mit der Testinstanz `abgabe.csv` ohne Fehler ausgeführt werden kann, dann wird nach dem Beenden des Programms im Verzeichnis `results` eine Ergebnisdatei mit dem Namen `solution-abgabe.csv` erzeugt. Beachten Sie dabei, dass Sie die gesamte Testinstanz ausführen und nicht nur eine Subinstanz.

Die Datei `solution-abgabe.csv` beinhaltet Zeitmessungen der Ausführung der Testinstanz `abgabe.csv`, welche in einem Webbrowser visualisiert werden können. Auch Ergebnisdateien anderer Testinstanzen können zu Testzwecken visualisiert werden. Öffnen Sie dazu die Datei `visualization.html` in Ihrem Webbrowser und klicken Sie rechts unten auf den Dateiauswahlknopf, um `solution-abgabe.csv` auszuwählen.

Beantworten Sie basierend auf der Visualisierung die Fragestellungen aus dem folgenden Abschnitt.

8 Fragestellungen

Öffnen Sie `solution-abgabe.csv` und bearbeiten Sie folgende Aufgaben- und Fragestellungen:

1. Finden Sie ein eigenes kleines Beispiel für die Farbenmenge $\{0, 1, 2\}$ und höchstens sieben Knoten, das zeigt, dass der Greedy-Algorithmus `greedyMCV` nicht immer optimale Lösungen findet. Fertigen Sie ein Bild dieses Gegenbeispiels an und begründen Sie Ihre Lösung. Wenn hilfreich, können Sie gerne die in `visualization.html` beinhaltete Graph-Plotting-Funktionalität (samt chronologischer Färbungsdemo) verwenden.

2. Betrachten Sie zuerst den oberen Plot, der die Laufzeit des Zählens der Verstöße zeigt. Welche asymptotische Laufzeitabhängigkeit von der Eingangsgröße lässt sich erkennen? Entspricht das Ihren

Erwartungen? Warum bzw. warum nicht? Speichern Sie im Anschluss den Plot als Bild, indem Sie in der Menüleiste rechts über dem Plot auf den Fotoapparat drücken.

3. Betrachten Sie nun den mittleren Plot, der die Lösungsqualität von `greedyMCV` und die Lösungsqualität von `greedyAlternativeMCV` zeigt. Bewerten Sie die Vorzüge der Algorithmen anhand der resultierenden Daten. Speichern Sie im Anschluss den Plot als Bild.

4. Betrachten Sie nun den unteren Plot, der die Laufzeit von `greedyMCV` und die Laufzeit von `greedyLookAheadMCV` zeigt. Bewerten Sie die Vorzüge der Algorithmen anhand der resultierenden Daten. Sie werden bemerken, dass `greedyLookAheadMCV` immer zumindest gleich gut wie `greedyMCV` abschneidet. Erklären Sie die Gründe dafür kurz. Speichern Sie im Anschluss den Plot als Bild.

Falls sich im Zuge der Evaluierung die Darstellung der Plots auf ungewünschte Weise verändert (z. B. durch die Auswahl eines zu kleinen Ausschnitts), können Sie mittels Doppelklick auf den Plot oder Klick auf das Haus in der Menüleiste die Darstellung zurücksetzen.

Fügen Sie Ihre Antworten in einem Bericht gemeinsam mit den erstellten Bildern der Visualisierung der Laufzeiten der Testinstanz `abgabe.csv` zusammen.

9 Abgabe

Laden Sie die Datei `task/src/main/java/StudentSolutionForExerciseImplementation.java` in der TUWEL-Aktivität *Hochladen Source-Code P2* hoch. Fassen Sie diesen Bericht mit den anderen für das zugehörige Abgabegespräch relevanten Berichten in einem PDF zusammen und geben Sie dieses in der TUWEL-Aktivität *Hochladen Bericht Abgabegespräch 1* ab.

10 Nachwort

Das Graph-Färbungsproblem gehört zu den klassischen NP-schweren Problemen und hat eine sehr umfangreiche Literaturlandschaft. Greedy-Algorithmen sind in diesem Zusammenhang besonders beliebt, weil sie für

dieses Problem sehr rasch Näherungslösungen liefern können. Allerdings können sie, wie Sie in der Aufgabe gesehen haben, oft auch suboptimale und sogar recht schlechte Lösungen produzieren, weshalb Sie im weiteren Verlauf der Vorlesung auch andere aber im Allgemeinen deutlich aufwändigere Algorithmen kennenlernen werden, die derartige Aufgaben besser bzw. optimal lösen können.

Graph-Färbung tritt in etwas flexibleren Formen in der Industrie allgegenwärtig auf: In der Compiler-Optimierung etwa wird Registerallokation als Färbungsproblem modelliert, in der Mobilfunkplanung die Frequenzzuweisung; weiters finden sich verwandte Problemstellungen in der Terminplanung oder Stundenplan-Generierung, wo es allgemein darum geht Ressourcen (z. B. Zeitfenster, Frequenzen, Räume, Mitarbeiter) so zuzuweisen, dass Konflikte möglichst minimiert werden. Greedy-Heuristiken sind dabei häufig der Einstieg, um in Echtzeit oder bei sehr großen Instanzen schnelle und qualitativ plausible Ergebnisse zu erzielen. Schon kleine algorithmische Fortschritte können in komplexen Systemen jedoch bedeutende Effekte erzielen. Verbesserte Färbungen führen in Anwendungen direkt zu weniger Konflikten, besserer Auslastung von Ressourcen, reduzierten Laufzeiten und damit letztlich zu Kosten- und Energieeinsparungen.

Wir hoffen, dass Ihnen diese Aufgabe einen spannenden Einblick in die diskrete Optimierung gibt. Falls Sie Interesse an weiteren Algorithmen zur Lösung schwieriger kombinatorischer Optimierungsaufgaben haben, freuen wir uns, Sie in unseren weiterführenden Lehrveranstaltungen zu begrüßen:

192.162 VU Heuristic Optimization Techniques
192.200 VU Mathematical Programming and Optimization in Transport Logistics
192.026 Effiziente Algorithmen